Ejemplo de Pruebas Unitarias con Mockito en una API REST con Spring Data JPA

En este ejemplo, vamos a probar un servicio de productos en una API REST con Spring Boot usando Mockito para simular el comportamiento del repositorio de datos.

Dependencias necesarias en pom.xml

Para ejecutar pruebas unitarias con JUnit 5 y Mockito, asegúrate de tener estas dependencias:

```
<dependencies>
  <!-- Dependencias para pruebas unitarias e integración -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
    <exclusions>
      <exclusion>
        <groupId>org.junit.vintage</groupId>
        <artifactId>junit-vintage-engine</artifactId>
      </exclusion>
    </exclusions>
  </dependency>

  <!-- Mockito -->
  <dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-core</artifactId>
    <scope>test</scope>
```

```
  </dependency>

  <!-- Mockito para Spring Boot -->
  <dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-junit-jupiter</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

1. Implementación de la API REST

La API REST maneja productos y tiene servicios y un repositorio en Spring Data JPA.

Entidad Producto

```
package com.miempresa.api.models;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "productos")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Producto {
  @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String nombre;

    private double precio;
}
```

Repositorio ProductoRepository

```
package com.miempresa.api.repositories;

import com.miempresa.api.models.Producto;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface ProductoRepository extends JpaRepository<Producto, Long> {
    Optional<Producto> findByNombre(String nombre);
}
```

Servicio ProductoService con logs

```
package com.miempresa.api.services;

import com.miempresa.api.models.Producto;
import com.miempresa.api.repositories.ProductoRepository;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;
import java.util.List;
```

```
import java.util.Optional;

@Service
@Slf4j
public class ProductoService {
  private final ProductoRepository productoRepository;

  public ProductoService(ProductoRepository productoRepository) {
    this.productoRepository = productoRepository;
  }

  public List<Producto> obtenerProductos() {
    log.info("Obteniendo lista de productos");
    return productoRepository.findAll();
  }

  public Producto guardarProducto(Producto producto) {
    log.info("Guardando producto: {}", producto.getNombre());
    return productoRepository.save(producto);
  }

  public Optional<Producto> obtenerProductoPorNombre(String nombre) {
    log.info("Buscando producto por nombre: {}", nombre);
    return productoRepository.findByNombre(nombre);
  }
```

```
  public void eliminarProducto(Long id) {
    log.warn("Eliminando producto con ID: {}", id);
    productoRepository.deleteById(id);
  }
}
```

2. Pruebas Unitarias con Mockito

Ahora probaremos el servicio ProductoService usando Mockito para simular el comportamiento del repositorio.

Prueba Unitaria: ProductoServiceTest.java

```
package com.miempresa.api.services;
import com.miempresa.api.models.Producto;
import com.miempresa.api.repositories.ProductoRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.Mockito;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Arrays;
import java.util.List;
import java.util.Optional;

import static org.assertj.core.api.Assertions.assertThat;
```

```
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class ProductoServiceTest {

  @Mock
  private ProductoRepository productoRepository;

  @InjectMocks
  private ProductoService productoService;

  private Producto producto1;
  private Producto producto2;

  @BeforeEach
  void setUp() {
    producto1 = new Producto(1L, "Laptop", 1500.99);
    producto2 = new Producto(2L, "Mouse", 25.50);
  }

  @Test
  void obtenerProductos_DeberiaRetornarListaDeProductos() {
    // Given (Dado que...)
    when(productoRepository.findAll()).thenReturn(Arrays.asList(producto1,
producto2));
```

```
    // When (Cuando se ejecuta el método)
    List<Producto> productos = productoService.obtenerProductos();

    // Then (Entonces se verifica el resultado)
    assertThat(productos).isNotNull();
    assertThat(productos.size()).isEqualTo(2);
    verify(productoRepository, times(1)).findAll();
}

@Test
void guardarProducto_DeberiaRetornarElProductoGuardado() {
    // Given
    when(productoRepository.save(any(Producto.class))).thenReturn(producto1);

    // When
    Producto productoGuardado = productoService.guardarProducto(producto1);

    // Then
    assertThat(productoGuardado).isNotNull();
    assertThat(productoGuardado.getNombre()).isEqualTo("Laptop");
    verify(productoRepository, times(1)).save(any(Producto.class));
}

@Test
void obtenerProductoPorNombre_DeberiaRetornarProducto() {
    // Given
```

```
when(productoRepository.findByNombre("Laptop")).thenReturn(Optional.of(producto1));

    // When
    Optional<Producto> productoEncontrado = productoService.obtenerProductoPorNombre("Laptop");

    // Then
    assertThat(productoEncontrado).isPresent();
    assertThat(productoEncontrado.get().getNombre()).isEqualTo("Laptop");
    verify(productoRepository, times(1)).findByNombre("Laptop");
  }

  @Test
  void eliminarProducto_DeberiaEliminarProductoPorId() {
    // Given
    Long id = 1L;
    doNothing().when(productoRepository).deleteById(id);

    // When
    productoService.eliminarProducto(id);

    // Then
    verify(productoRepository, times(1)).deleteById(id);
  }
}
```

Explicación de las Pruebas

1. Configuración con @Mock y @InjectMocks
    - @Mock simula la dependencia ProductoRepository.
    - @InjectMocks inyecta el mock en ProductoService.
2. Método obtenerProductos()
    - Simula que productoRepository.findAll() devuelve una lista de productos.
    - Verifica que la lista tiene 2 elementos y que findAll() fue llamado exactamente 1 vez.
3. Método guardarProducto()
    - Simula la persistencia de un producto en la base de datos.
    - Verifica que el producto devuelto no es null y que el nombre es "Laptop".
4. Método obtenerProductoPorNombre()
    - Simula la búsqueda de un producto por nombre.
    - Verifica que el resultado es presente (isPresent()) y que el nombre coincide.
5. Método eliminarProducto()
    - Simula la eliminación de un producto por ID.
    - Verifica que deleteById(id) fue llamado una vez.

Ejecución de las Pruebas

Para ejecutar las pruebas, usa: mvn test

o desde IntelliJ/Eclipse, ejecuta ProductoServiceTest.java.

Resultado esperado: Todas las pruebas deben pasar con éxito.

Conclusión

Se crearon pruebas unitarias con Mockito en Spring Boot.
Se simularon llamadas a Spring Data JPA con when(...) y doNothing().
Se usaron assertions para validar los resultados.
Se verificó que los métodos de ProductoRepository fueron llamados correctamente.

Siguientes pasos:

- Agregar pruebas de integración con MockMvc.
- Incluir Testcontainers para pruebas con PostgreSQL real.